



STROJNO UČENJE

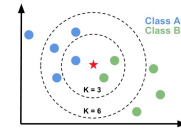
11. predavanje

Hiperparametri i cjevovodi

AI Center Lipik
THE FUTURE IS HERE

11.1 Podešavanje hiperparametara - GridSearchCV

- Parametri – podešavaju se automatski tokom treniranja
- Hiperparametri – mi ih podešavamo prije treniranja

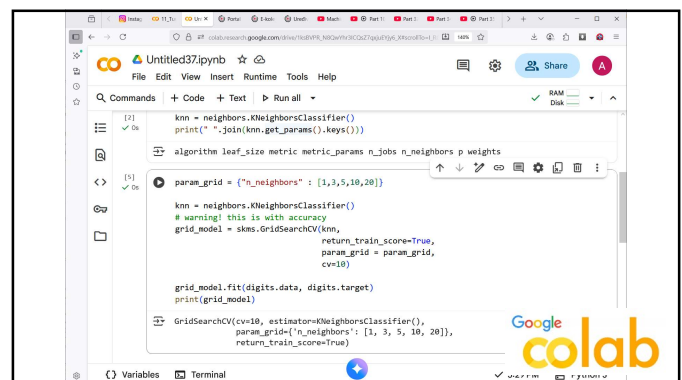


Google
colab

```
knn = neighbors.KNeighborsClassifier()
print(" ".join(knn.get_params().keys()))

algorithm leaf_size metric metric_params n_jobs n_neighbors p weights
```

Google
colab

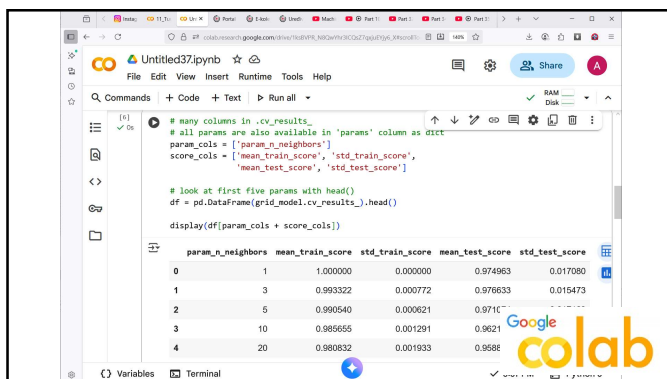


```
param_grid = {"n_neighbors": [1, 3, 5, 10, 20]}

knn = neighbors.KNeighborsClassifier()
# warning! this is with accuracy
grid_model = sks.GridSearchCV(knn,
                              return_train_score=True,
                              param_grid=param_grid,
                              cv=10)

grid_model.fit(digits.data, digits.target)
print(grid_model)

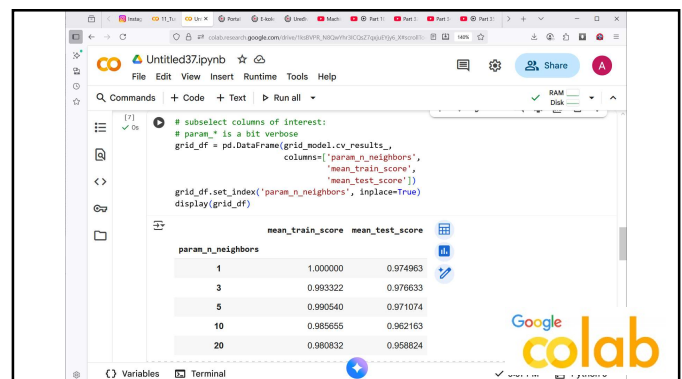
GridSearchCV(cv=10, estimator=KNeighborsClassifier(),
             param_grid={'n_neighbors': [1, 3, 5, 10, 20]},
             return_train_score=True)
```



```
# many columns in .cv_results_
# all params are also available in 'params' column as dict
param_cols = ['param_n_neighbors']
score_cols = ['mean_train_score', 'std_train_score',
              'mean_test_score', 'std_test_score']

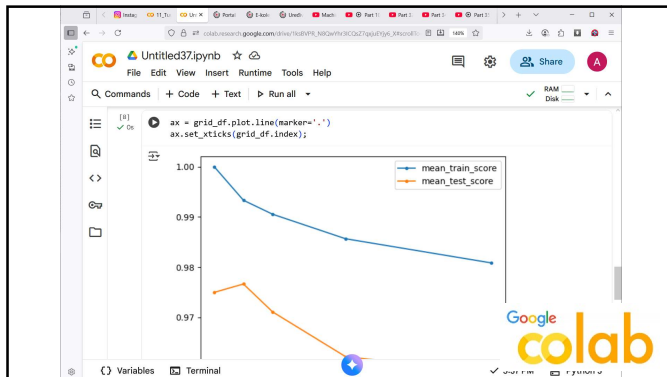
# look at first five params with head()
df = pd.DataFrame(grid_model.cv_results_.head())
display(df[param_cols + score_cols])
```

	param_n_neighbors	mean_train_score	std_train_score	mean_test_score	std_test_score
0	1	1.000000	0.000000	0.974963	0.017080
1	3	0.993322	0.000772	0.976633	0.015473
2	5	0.990540	0.000621	0.971074	
3	10	0.985655	0.001291	0.962163	
4	20	0.980832	0.001933	0.958824	



```
# subselect columns of interest:
# param_ is a bit verbose
grid_df = pd.DataFrame(grid_model.cv_results_,
                       columns=['param_n_neighbors',
                                'mean_train_score',
                                'mean_test_score'])
grid_df.set_index('param_n_neighbors', inplace=True)
display(grid_df)
```

	param_n_neighbors	mean_train_score	mean_test_score
1	1	1.000000	0.974963
3	3	0.993322	0.976633
5	5	0.990540	0.971074
10	10	0.985655	0.962163
20	20	0.980832	0.958824



```
param_grid = {"n_neighbors" : np.arange(1,11),
              "weights"      : ['uniform', 'distance'],
              "p"            : [1,2,4,8,16]}

knn = neighbors.KNeighborsClassifier()
grid_model = skms.GridSearchCV(knn, param_grid = param_grid, cv=10)
```

```
# %timeit -r1 grid_model.fit(digits.data, digits.target)
%timeit -r1 grid_model.fit(iris.data, iris.target)
```

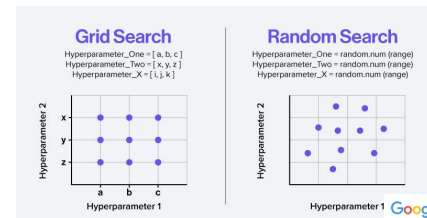
```
param_df = pd.DataFrame.from_records(grid_model.cv_results_['params'])
param_df['mean_test_score'] = grid_model.cv_results_['mean_test_score']
param_df.sort_values(by='mean_test_score').tail()
```

	n_neighbors	p	weights	mean_test_score
95	10	4	distance	0.98
96	10	8	uniform	0.98
97	10	8	distance	0.98
98	10	16	uniform	0.98
99	10	16	distance	0.98

```
print("Best Estimator:", grid_model.best_estimator_,
      "Best Score:",    grid_model.best_score_,
      "Best Params:",   grid_model.best_params_, sep="\n")
```

```
Best Estimator:
KNeighborsClassifier(n_neighbors=np.int64(8), p=4)
Best Score:
0.9800000000000001
Best Params:
{'n_neighbors': np.int64(8), 'p': 4, 'weights': 'uniform'}
```

11.2. Podešavanje hiperparametara - RandomizedSearchCV



```

import scipy.stats as ss
knn = neighbors.KNeighborsClassifier()
param_dists = {"n_neighbors": ss.randint(1,11), # values from [1,10]
               "weights": ["uniform", "distance"],
               "p": ss.geom(p=.3)}

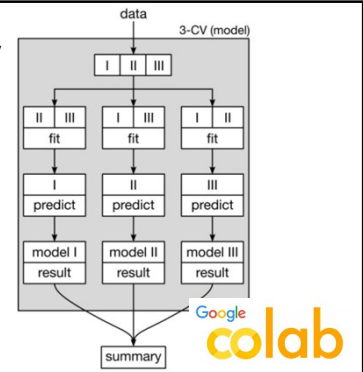
mod = skes.RandomizedSearchCV(knn,
                             param_distributions = param_dists,
                             cv=10,
                             n_iter=20) # how many times do we sample?

# fitting 20 models
%timeit -r1 mod.fit(iris.data, iris.target)
print(mod.best_score_)

535 ms ± 0 ns per loop (mean ± std. dev. of 1 run, 1 loop each)
0.9800000000000001

```

11.3. Ugnježena CV



```

param_grid = {"n_neighbors": np.arange(1,11),
              "weights": ["uniform", "distance"],
              "p": [1,2,4,8,16]}

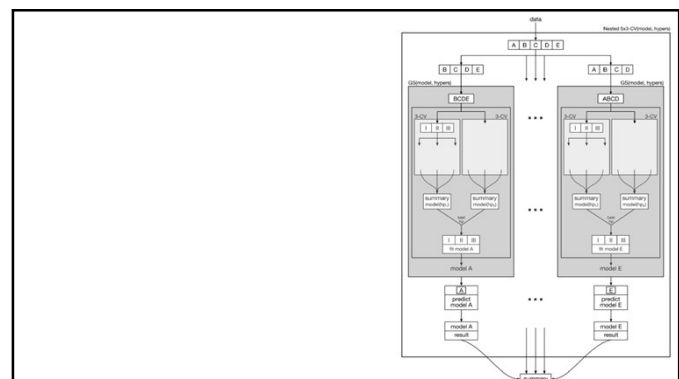
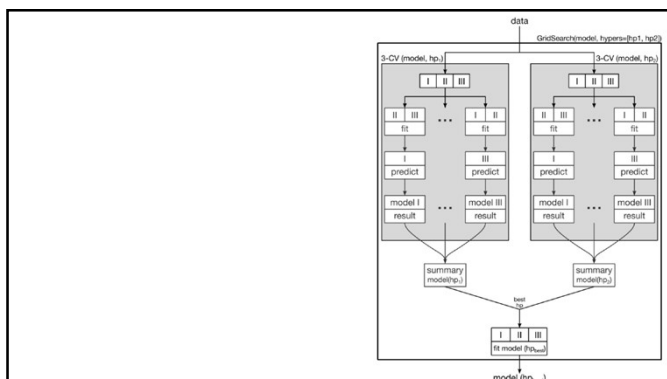
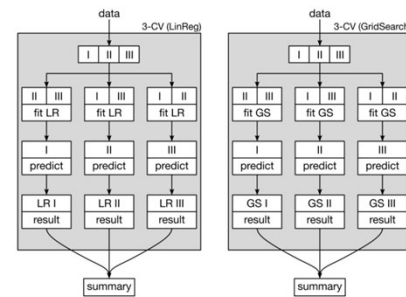
knn = neighbors.KNeighborsClassifier()
grid_knn = skes.GridSearchCV(knn,
                             param_grid = param_grid,
                             cv=3)

print(grid_knn)

GridSearchCV(cv=3, estimator=KNeighborsClassifier(),
             param_grid={'n_neighbors': array([ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]),
                         'p': [1, 2, 4, 8, 16]},
             weights=['uniform', 'distance'])

```

11.3.1 GridSearch kao model ?!



```

param_grid = {"n_neighbors": np.arange(1,11),
              "weights": ['uniform', 'distance'],
              "p": [1,2,4,8,16]}

knn = neighbors.KNeighborsClassifier()
grid_knn = GridSearchCV(knn,
                       param_grid=param_grid,
                       cv=5)

print(grid_knn)

GridSearchCV(cv=5, estimator=KNeighborsClassifier(),
             param_grid={'n_neighbors': array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]),
                       'p': [1, 2, 4, 8, 16],
                       'weights': ['uniform', 'distance']})

outer_scores = sks.cross_val_score(grid_knn,
                                     iris.data, iris.target,
                                     cv=5)

print(outer_scores)

[0.96666667 1. 0.93333333 0.96666667 1.]

```

[0.96666667 1. 0.93333333 0.96666667 1.]

```

param_grid = param_grid,
cv=2)

outer_scores = sks.cross_val_score(grid_knn,
                                     iris.data,
                                     iris.target,
                                     cv=5)

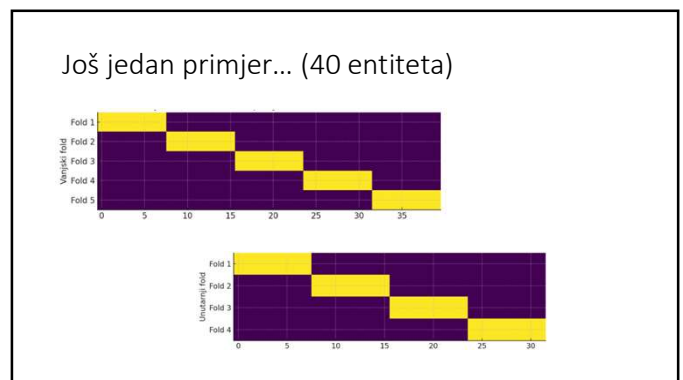
# how does this do over all??
print(outer_scores)

[0.96666667 0.96666667 0.93333333 0.96666667 1.]

grid_knn.fit(iris.data, iris.target)
preferred_params = grid_knn.best_estimator_.get_params()
final_knn = neighbors.KNeighborsClassifier(**preferred_params)
final_knn.fit(iris.data, iris.target)

KNeighborsClassifier
KNeighborsClassifier(n_neighbors=np.int64(7), p=4, weights='distance')

```



Primjer koji mi je dao GPT 5.0 model



Evo kompletna, samostalan primjer s scikit-learn datasetom Adult (income) koji pokazuje ugniježđeni (nested) CV: unutra je GridSearchCV (tuning), vani je StratifiedKFold (poštena procjena). Uključio sam i OneHotEncoder za kategorijske, imputaciju za NaN-ove te StandardScaler za numeričke.



```

# Nested CV primjer na Adult (income) skupu podataka
# - Unutarnja CV: GridSearchCV (tuning hiperparametara)
# - Vanjska CV: StratifiedKFold (nepriistrana procjena nakon tuninga)

import numpy as np
import pandas as pd

from sklearn.datasets import fetch_openml
from sklearn.model_selection import StratifiedKFold, GridSearchCV, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

# 1) Učitaj Adult (preko scikit-learn-a)
adult = fetch_openml("adult", version=2, as_frame=True)
X = adult.data
y = adult.target

# Target je string ('<=50K', '>50K'); pretvorimo u 0/1
y = (y == ">50K").astype(int)

# 2) Podjela kolona po tipu
num_cols = X.select_dtypes(include=["number"]).columns.tolist()
cat_cols = X.select_dtypes(exclude=["number"]).columns.tolist()

```

```
# 3) Pretprocesiranje:
# - numeričke: imputacija medianom + skaliranje
# - kategorijske: imputacija najčešćom + OneHotEncoder (drop='first' + handle_unknown)
preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imp", SimpleImputer(strategy="median")),
            ("sc", StandardScaler())
        ]), num_cols),
        ("cat", Pipeline([
            ("imp", SimpleImputer(strategy="most_frequent")),
            ("oh", OneHotEncoder(drop="first", handle_unknown="ignore"))
        ]), cat_cols),
    ],
    remainder="drop"
)

# 4) Cijeli pipeline: preprocessor + klasifikator
pipe = Pipeline([
    ("pre", preprocessor),
    ("clf", LogisticRegression(
        solver="liblinear", # stabilno za binarni problem
        max_iter=1000
    ))
])
```

```
# 5) Grid za tuning unutar unutarnje CV
param_grid = {
    "clf_C": [0.01, 0.1, 1, 10],
    "clf_class_weight": [None, "balanced"]
}

# 6) Unutarnja i vanjska CV
inner_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=1) # za GridSearchCV
outer_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42) # za nested evaluaciju

# 7) GridSearchCV (unutarnja CV)
gs = GridSearchCV(
    estimator=pipe,
    param_grid=param_grid,
    scoring="roc_auc",
    cv=inner_cv,
    n_jobs=-1
)

# 8) Brza nested evaluacija: cross_val_score nad GridSearchCV (vanjska CV)
outer_scores = cross_val_score(gs, X, y, cv=outer_cv, scoring="roc_auc", n_jobs=-1)
print(f"Nested CV AUC: {outer_scores.mean():.4f} +/- {outer_scores.std():.4f}")
```

11.4. Pipelines (cjevovodi?)



Jednostavni pipeline

```
from sklearn import preprocessing as skpre
from sklearn import linear_model
from sklearn import pipeline
+from sklearn import model_selection as skms

scaler = skpre.StandardScaler()
logreg = linear_model.LogisticRegression()

pipe = pipeline.make_pipeline(scaler, logreg)
print(skms.cross_val_score(pipe, iris.data, iris.target, cv=10))

[1. 0.93333333 1. 1. 0.93333333 0.93333333
 0.8 1. 1. 1. ]
```

```
def extract_name(obj):
    return str(logreg.__class__).split('.')[1][:-2].lower()

print(logreg.__class__)
print(extract_name(logreg))

<class 'sklearn.linear_model._logistic.LogisticRegression'>
logisticregression

pipe.named_steps.keys()

dict_keys(['standardscaler', 'logisticregression'])
```

```
pipe = pipeline.Pipeline(steps=[('scaler', scaler),
                                ('knn', knn)])

cv_scores = skms.cross_val_score(pipe, iris.data, iris.target,
                                cv=10,
                                n_jobs=-1) # all CPUs

print(pipe.named_steps.keys())
print(cv_scores)

dict_keys(['scaler', 'knn'])
[1. 0.93333333 1. 0.93333333 0.86666667 0.93333333
 0.86666667 1. 1. 1.]
```

Složeniji pipeline (koraci...)

- 1. Standardiziranje podataka
- 2. Stvaranje uvjeta interakcije između značajki,
- 3. Diskretizacija značajki na velike i male
- 4. Treniranje

Kreiranje „našeg” transformera

- Kreiramo klasu baziranu na klasi TransformerMixin

```
from sklearn.base import TransformerMixin
class Median_Big_Small(TransformerMixin):
    def __init__(self):
        pass
    def fit(self, ftrs, tgt=None):
        self.medians = np.median(ftrs)
        return self
    def transform(self, ftrs, tgt=None):
        return ftrs > self.medians
```

```
scaler = skpre.StandardScaler()
quad_inters = skpre.PolynomialFeatures(degree=2,
                                       interaction_only=True,
                                       include_bias=False)

median_big_small = Median_Big_Small()
knn = neighbors.KNeighborsClassifier()

pipe = pipeline.Pipeline(steps=[('scaler', scaler),
                                ('inter', quad_inters),
                                ('mbs', median_big_small),
                                ('knn', knn)])

cv_scores = skms.cross_val_score(pipe, iris.data, iris.target, cv=10)

print(cv_scores)
```

[0.6	0.66666667	0.8	0.73333333	0.8	0.66666667
0.66666667	0.66666667	0.8	0.8		

11.5. Cjevovodi i podešavanje zajedno

```
# create pipeline components and pipeline
scaler = skpre.StandardScaler()
poly = skpre.PolynomialFeatures()
lasso = linear_model.Lasso(selection='random', tol=.01)
pipe = pipeline.make_pipeline(scaler,
                              poly,
                              lasso)

# specified hyper-parameters to compare
param_grid = {"polynomialfeatures__degree" : np.arange(2,6),
              "lasso__alpha" : np.logspace(1,6,6,base=2)}

from pprint import pprint as pp
pp(param_grid)

{'lasso__alpha': array([ 2.,  4.,  8., 16., 32., 64.]),
 'polynomialfeatures__degree': array([2, 3, 4, 5])}
```

```
mod = skms.GridSearchCV(pipe, param_grid, n_jobs=-1)
mod.fit(diabetes.data, diabetes.target)
```

Show hidden output

```
import textwrap
for name, step in mod.best_estimator_.named_steps.items():
    print("Step:", name)
    print(textwrap.indent(textwrap.fill(str(step), 50), " " * 6))

Step: standardscaler
StandardScaler()
Step: polynomialfeatures
PolynomialFeatures(degree=np.int64(2))
Step: lasso
Lasso(alpha=np.float64(4.0), selection='random',
      tol=0.01)

pp(mod.best_params_)

{'lasso__alpha': np.float64(4.0), 'polynomialfeatures__degree': np.int64(2)}
```